

ADVANCED PROGRAMMING TECHNIQUES · TEAM 10 · WASTED

Smart Waste Bin

An End-to-End IoT System

PIR-sensor

GAS-sensor

Node-RED

Python-dashboard

Home Assistant

JSON-LD

Flask

MQTT

Docker

HIVEMQ

Raspberry Pi 5

Vasileios Banakos · Charalampos Papadopoulos · Ioanna Trochatou

What We Set Out to Build

A real hardware-to-dashboard IoT pipeline — every component decoupled through MQTT.



Real Hardware

Raspberry Pi 5 reads PIR motion & MQ-3 gas from physical sensors — no simulations.



End-to-End Pipeline

From a GPIO interrupt all the way to a live dashboard, every hop is tracked.



Loose Coupling

Every component speaks only MQTT. None talks directly to another — swap any service freely.



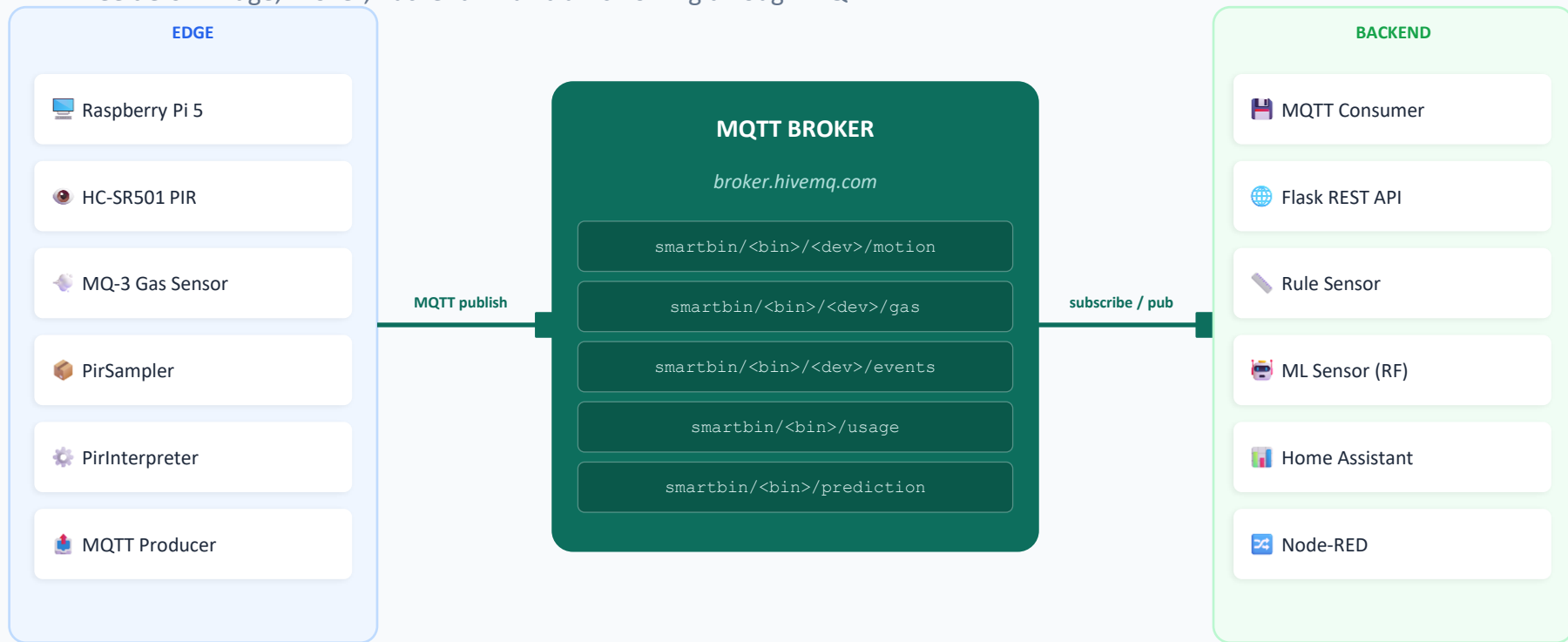
Layered Intelligence

Rule-based classification and a RandomForest that predicts whether the next hour will be busy.

Stack: Raspberry Pi 5 · MQTT / HiveMQ · Flask REST API · JSONL log · RandomForest · Home Assistant · Tkinter · Docker Compose

System Architecture

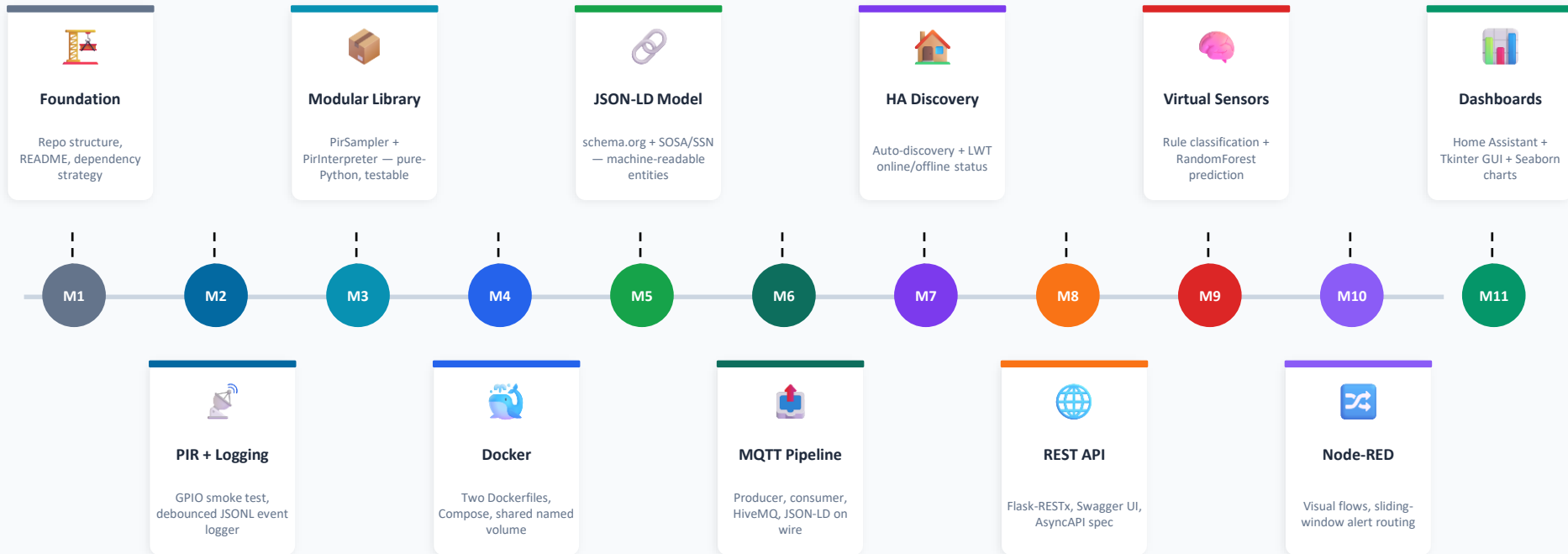
Three tiers — Edge, Broker, Backend — all traffic flowing through MQTT.



Key design principle: every component is "blind" to the others — it only knows MQTT topics. Swap or restart any service without touching the rest.

11 Milestones

Each milestone adds exactly one layer on top of the previous. Nothing was skipped.



Repository tree-What Lives Where

Every folder maps to one or more milestones. Colour = owning milestone tier.

Project Files

smart-waste-bin/

pi_edge_node/	M2	M3	M4
pir_smoke_test.py	M2		
pir_event_logger.py	M2		
motion_sensor_lib/	M3		
pir_sampler.py	M3		
pir_interpreter.py	M3		
pir_mqtt_producer.py	M6	M7	
Dockerfile	M4		
src/	M4	M6	M8
mqtt_consumer.py	M6		
api.py	M8		
virtual_sensor_rules.py	M9		
virtual_sensor_ml.py	M9		
train_model.py	M9		
Dockerfile	M4		
models/	M5		
context.jsonld	M5		
wastebin.jsonld	M5		
sensor.jsonld	M5		
environment.jsonld	M5		

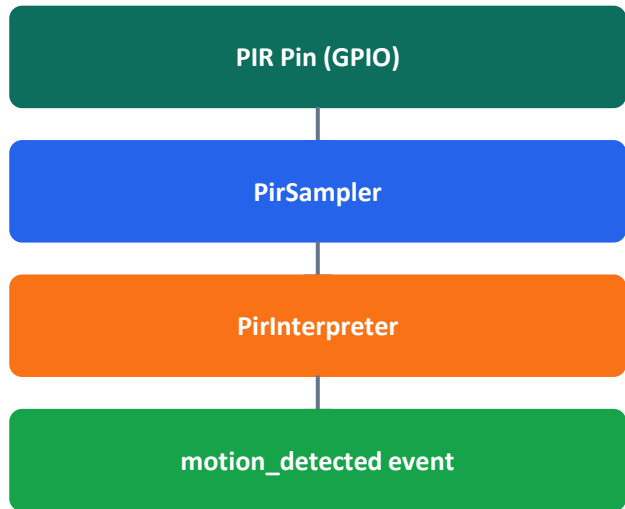
More Files

home_assistant/	M7	M11
configuration.yaml	M7	
dashboard.yaml	M11	
node_red/	M10	
flows.json	M10	
laptop_dashboard/	M11	
mqtt_gui_consumer.py	M11	
analyze.py	M11	
docs/	M8	
asyncapi.yaml	M8	
docker-compose.yml	M4	
requirements.txt	M1	
README.md	M1	

Edge Intelligence

M2 · M3

How the library works



Zero hardware deps in the interpreter → unit-test on any laptop ✓

PirSampler — raw GPIO reader

- ▶ Only reads HIGH/LOW state of the pin.
- ▶ Knows nothing about events or timing.
- ▶ Reused by logger, debug tool, and producer.

PirInterpreter — state machine

- ▶ `min_high_s`: ignores electrical spikes shorter than threshold.
- ▶ `cooldown_s`: rate-limits repeated triggers.
- ▶ Converts noisy raw stream into clean events.
- ▶ Pure Python — no `gpiozero` import.

Why this split matters

- ▶ Hardware bugs isolated to PirSampler.
- ▶ Logic bugs isolated to PirInterpreter.
- ▶ CI can run the interpreter tests without a Pi.

Structured Data — JSON-LD + MQTT Topic Scheme

M5 · M6

Why JSON-LD?

No magic strings

Every field links to schema.org or SOSA/SSN — not a private convention.

Events as objects

Each motion event is a schema:Event with startDate, madeBySensor, hasSimpleResult.

Self-describing

Any IoT agent that reads JSON-LD can interpret our data without our docs.

Virtual sensors first-class

Derived observations sit next to physical ones in the model.

MQTT Topic Hierarchy

Topic	Payload	By
.../motion	detected / clear	edge
.../gas	detected / clear	edge
.../events	rich JSON-LD	edge
.../event_count	integer (retained)	edge
.../last_motion	ISO timestamp	edge
.../online	true/false (LWT)	edge
.../usage	idle/low/med/high	rules
.../prediction	busy/quiet + conf%	ML
.../alert	JSON alert object	Node-RED

Virtual Sensors — From Raw Data to Decisions

M9



Rule-Based Sensor

How it works

Counts detections in a rolling time window, then maps the count to a usage level. No ML needed — pure Python logic.

Usage levels:

Idle

0 detections

Low

1–3

Medium

4–8

High

9+

→ Published to `smartbin/bin-01/usage`

→ Visible in Home Assistant + REST /virtual endpoint



ML-Based Sensor (RandomForest)

Training features

Hour of day

Day of week

Weekend flag

Output

Predicts whether the next hour will be busy or quiet — with a confidence percentage published alongside the label.

Example output

```
{ "prediction": "busy", "confidence": 0.79,  
  "model": "random_forest_v1" }
```

→ Published to `smartbin/bin-01/prediction`

→ Train with: `docker compose run --rm trainer`

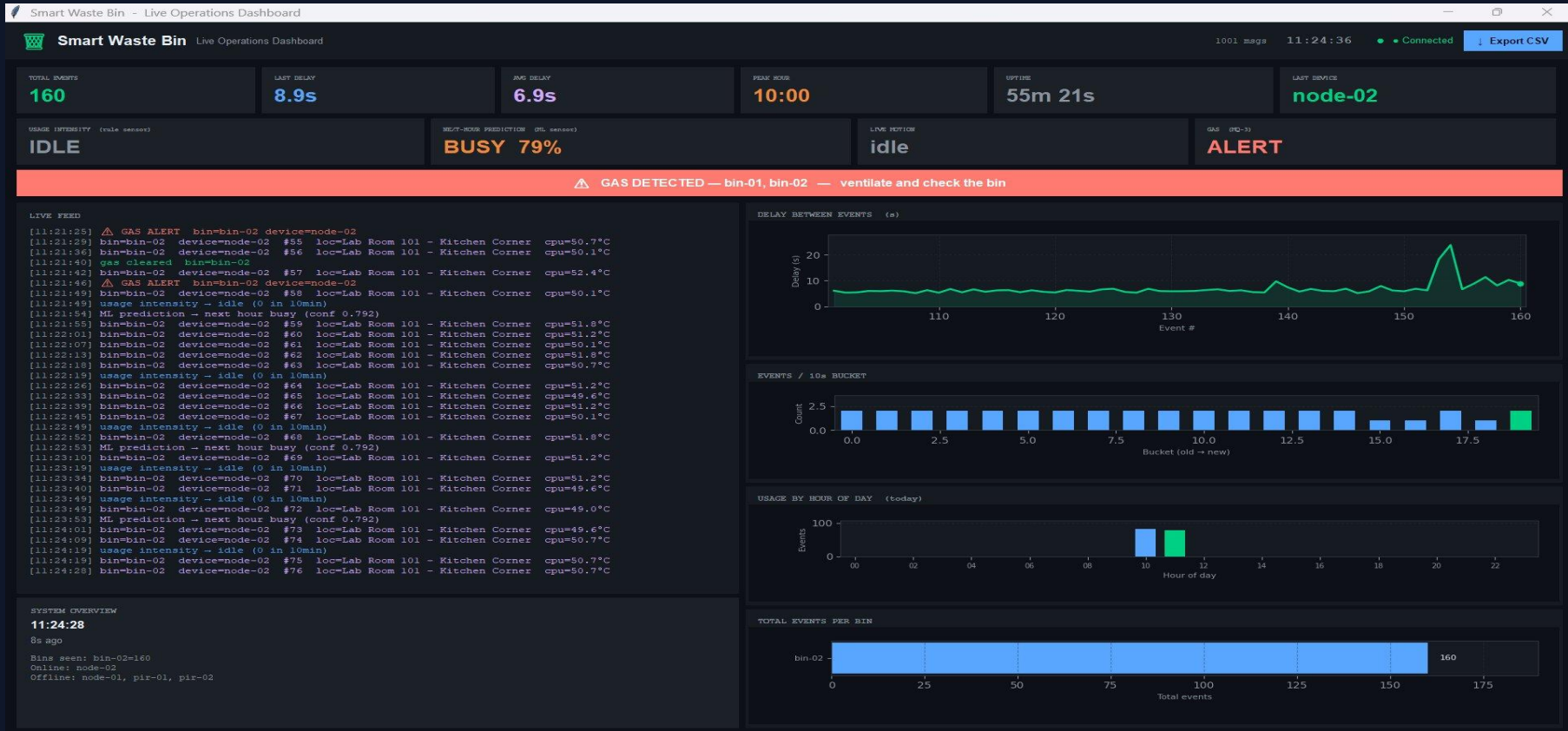
Both virtual sensors publish back to MQTT — treating AI predictions as "just another sensor reading." Neither knows the other exists.



Live Desktop Dashboard

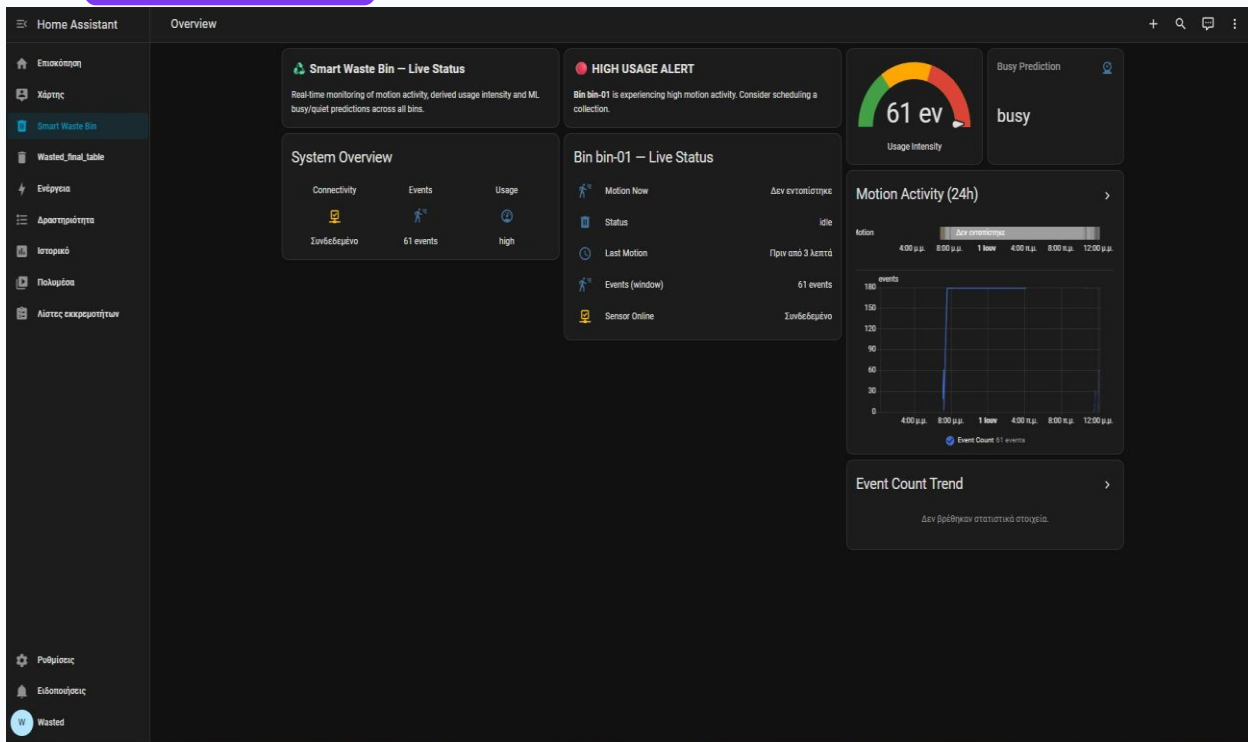
M11

Tkinter + Matplotlib · KPIs · Live event feed · 4 real-time charts · Gas alert banner · Online indicator · Demo mode: `python mqtt_gui_consumer.py --demo`



Home Assistant — Auto-Discovered Live Status

M7 + M11



Zero YAML config

Producer publishes MQTT discovery on connect — entities appear automatically.

Last Will & Testament

Broker delivers retained online=false the moment the Pi drops off the network.

ML prediction card

Shows busy/quiet prediction with confidence — updated every minute by the ML virtual sensor.

Usage intensity gauge

Real-time idle/low/medium/high from the rule-based virtual sensor.

REST API + Node-RED Low-Code Layer

M8

M10

Swagger UI — Auto-generated from Flask-RESTx

Smart Waste Bin API

[Base URL: /]
swagger.json

Full-based REST API for Smart Waste Bin data, sensors and MQTT bridge.

bins

Wastebin operations

GET	/bins/	List all registered bins
GET	/bins/{bin_id}	Get one bin
POST	/bins/{bin_id}/emptied	Record that a bin was emptied and publish status to MQTT
GET	/bins/{bin_id}/events	Get motion events for a bin (newest first)

sensors

Sensor operations

virtual

Virtual sensors

mqtt

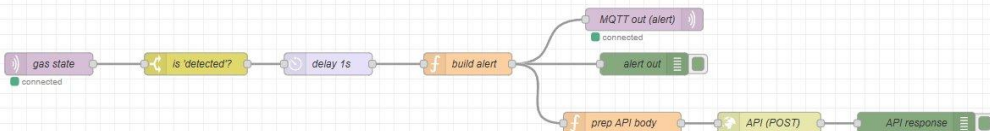
MQTT broker bridge

health

Health check

Models

Node-RED Flow — Gas alert → MQTT out + REST API POST



AsyncAPI spec in [docs/asyncapi.yaml](#) covers the MQTT surface.

API endpoints

GET	/bins/	List all registered bins
GET	/bins/{id}	Get one bin's details
POST	/bins/{id}/emptied	Record emptied + publish MQTT
GET	/bins/{id}/events	Motion events, newest first
GET	/sensors/	All registered sensors
GET	/virtual/	Virtual sensor readings
GET	/mqtt/status	Broker connection status
GET	/health	Health check

What Went Wrong — and What We Learned

01

Identity inconsistency

⚠️ Producer used `bin-group10`, rule sensor listened for `bin-01`, HA registered `bin_1`. Data silently vanished for hours.

✓ Single identity (`bin-01` / `pir-01`) enforced everywhere — env vars, CLI flags, YAML, code.

03

Layer caches lie

⚠️ A 3-second rebuild felt reassuring but was just reusing cached layers — code changes weren't picked up.

✓ `docker compose build --no-cache` is the only way to guarantee a clean build.

02

Docker build context

⚠️ Backend image accidentally built with the edge node's Dockerfile — `/app/src` was never copied. Training failed with 'can't open file'.

✓ `docker compose run --rm <svc> ls /app` to verify image contents before assuming.

04

Schema agreement

⚠️ REST API filtered events by a field the producer never emitted — `/events` returned an empty list for days.

✓ A shared schema document (this report) describes every field. Producer and consumer now agree.

What We Built

- Real hardware sensing on Raspberry Pi 5
- Decoupled MQTT messaging through HiveMQ
- Persistent JSONL event log
- Documented REST API with Swagger UI
- Rule-based and ML virtual sensors
- Visual Node-RED flows + alert routing
- Three visualisation surfaces
- One-command docker compose up deployment

Natural Next Steps



Ultrasonic fill-level sensor for true capacity tracking



Retrain ML on the real accumulated event log (not synthetic)



Migrate JSONL → InfluxDB for time-series queries



Add OAuth2 to the broker and REST API



Scale to multiple bins with no code changes

Thank You!

Questions?
